Mergesort

Divide, sort, merge – the N log N sort

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026

By the end you can

1. explain mergesort: divide, sort each half recursively, merge
2. implement `merge()` with an auxiliary array and trace it with `i`, `j`, `k`
3. implement the recursive `sort(a, lo, hi)` and trace its calls
4. argue that mergesort is correct
5. derive its cost: $2N \lg N$ data moves, $\frac{1}{2} N \lg N$ to $N \lg N$ compares
6. confirm $N \log N$ growth with a doubling test – mergesort scales
7. name its price, an extra array, and reduce other problems to sorting

1,000 exam papers, two friends

How do you split the work?

Each friend sorts half – then you merge

Look at the top paper of each sorted pile, take the one that comes first, repeat.

With insertion sort, how long does half the pile take? → A quarter of the time: $(N/2)^2 = N^2/4$.
→ Two halves: half the time. Merging: one look per paper.

Now do the same for each half again, and again . . .

Mergesort: the idea

- **Divide** the array into two halves
- **Sort** each half – recursively
- **Merge** the two sorted halves into a sorted whole
- Base case: 0 or 1 entries are sorted
- **Divide-and-conquer**: independent parts, solved independently, then combined

input

```
was had him and | you his the but
```

sort left, sort right

```
and had him was | but his the you
```

merge

```
and but had him his the was you
```

John von Neumann (1903–1957) wrote mergesort in 1945, to test a new computer on a task other than numerical calculation.

Merging needs an extra array

- Merging two sorted halves inside the same array (**in place**) is very hard
- So `merge()` writes into an **auxiliary array** `aux[]` – and copies it back
- From outside it looks as if the merge happened in place
- The price: an extra array of size N (quicksort and heapsort sort in place – COSC 3210)

Half-open intervals, as in binary search

`a[lo, mid)` = `a[lo] ... a[mid-1]` `a[mid, hi)` = `a[mid] ... a[hi-1]`

Code-along 1: `Merge.java` in five steps

`Merge.java`

1	`private static String[] aux;`	the extra array
2	`merge(a, lo, mid, hi)`	fill aux[0, N), copy back
3	`sort(a)`	allocate aux once, start
4	`sort(a, lo, hi)`	base case, two calls, merge
5	`main()`	test client from Insertion

merge: two sorted halves
had was | and him
→ and had him was

sort: split, sort each half, merge
first run after step 5

```
$ java Merge < names16.txt   →   alice bob carlos . . .  wendy
```

Later: `TestMerge.java` – the doubling test from `TestInsertion`, with `Merge.sort(a)`

Merge: merge()

→ **Switch to editor · code-along 1**

1. Create **`Merge.java`** with a static field **`String[] aux`**
2. Write **`merge(a, lo, mid, hi)`**: fill `aux[0, N)`, copy back

data: `birkenkrahe.github.io/data/names16.txt` · classpath: `.:stdlib.jar`

merge(): one index per half, one into aux[]

```
private static String[] aux;
public static void merge(String[] a,
                         int lo, int mid, int hi)
{
    int i = lo, j = mid, N = hi - lo;
    for (int k = 0; k < N; k++)
    {
        if      (i == mid) aux[k] = a[j++];
        else if (j == hi)  aux[k] = a[i++];
        else if (a[j].compareTo(a[i]) < 0)
                           aux[k] = a[j++];
        else               aux[k] = a[i++];
    }
    for (int k = 0; k < N; k++)
        a[lo + k] = aux[k];
}
```

- i: left half
- j: right half
- k: next free place in aux
- left used up: take right
- right used up: take left
- else the smaller one
- then copy back

Trace: one merge, one row per step

			left half a[0, 4)				right half a[4, 8)				
i	j	k	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	aux[k]
0	4	0	**and**	had	him	was	**but**	his	the	you	and
1	4	1	and	**had**	him	was	**but**	his	the	you	but
1	5	2	and	**had**	him	was	but	**his**	the	you	had
2	5	3	and	had	**him**	was	but	**his**	the	you	him
3	5	4	and	had	him	**was**	but	**his**	the	you	his
3	6	5	and	had	him	**was**	but	his	**the**	you	the
3	7	6	and	had	him	**was**	but	his	the	**you**	was
4	7	7	and	had	him	was	but	his	the	**you**	you

bold: a[i] and a[j], compared **teal**: taken into aux[k] grey: already in aux

How many compares? → 7: none in the last row – the left half is used up.
How many data moves? → 8 into aux, 8 back: $2N = 16$.

companion card 1: birkenkrahe.github.io/demos/merge-sort.html